

Salient Object Detection using CNN

Fatbardh Fetoshi

1. Introduction

In this project, I focused on a computer vision problem called Salient Object Detection (SOD). The goal of this task is to identify the most important object in an image and separate it from the background.

This type of problem is useful in areas like image understanding, object recognition, and preparing images for other machine learning models.

To approach this, I built a Convolutional Neural Network (CNN) that takes an image as input and outputs a mask highlighting the most important regions.

2. Dataset

For this project, I used the ECSSD dataset, which contains images together with ground truth masks. These masks show which parts of the image are considered important.

Before training the model, I split the dataset into three parts: a training set, a validation set, and a test set. The training set was used to train the model, the validation set helped monitor performance during training, and the test set was used for the final evaluation. This ensures that the model is tested on images it has not seen before.

3. Data Preprocessing

Before training, I prepared the data to make it suitable for the model. All images were resized to a fixed size, and their pixel values were normalized between 0 and 1. The masks were converted into binary format, where 0 represents the background and 1 represents the object.

To improve the model's performance, I also applied some basic data augmentation techniques such as horizontal flipping, small rotations, and brightness adjustments. This helps the model generalize better instead of memorizing the training data.

4. Model Architecture

I implemented a CNN-based encoder-decoder architecture. The encoder part extracts important features from the image, while the decoder reconstructs these features into a saliency mask.

During this process, the model first reduces the image size to learn high-level features and then increases it again to produce the final output. The output is a single-channel image where the white areas represent the detected object.

5. Training Process

The model was trained using a combination of Binary Cross Entropy (BCE) loss and Dice loss. BCE helps measure pixel-level differences, while Dice loss focuses on the overlap between the predicted mask and the ground truth.

I used the Adam optimizer with a learning rate of 0.001 and a batch size of 2. I trained the model for two different settings, 15 epochs and 20 epochs, to compare the results. During training, I saved the best model based on the validation loss.

6. Evaluation

After training, I evaluated the model using several metrics. These included IoU, Precision, Recall, F1-score, and MAE. Each of these helps measure a different aspect of performance, such as accuracy, coverage, and overall error.

7. Results

After running the experiments, I obtained the following results.

For the model trained for 15 epochs, the IoU was 0.4311, precision was 0.5692, recall was 0.7081, F1-score was 0.6078, and MAE was 0.2415.

For the model trained for 20 epochs, the IoU slightly increased to 0.4338, precision decreased slightly to 0.5502, recall improved to 0.7413, F1-score was 0.6082, and MAE increased slightly to 0.2509.

8. Discussion

From these results, I noticed that increasing the number of epochs had only a small effect on performance. The recall improved, meaning the model was able to detect more

of the object, but the precision decreased slightly, which means it also included more background areas.

Overall, IoU and F1-score showed small improvements. This indicates a trade-off between precision and recall. The model performs reasonably well but still struggles with accurately capturing object boundaries.

9. Visualization

To better understand the model's performance, I visualized the results using the original image, ground truth mask, predicted mask, and overlay.

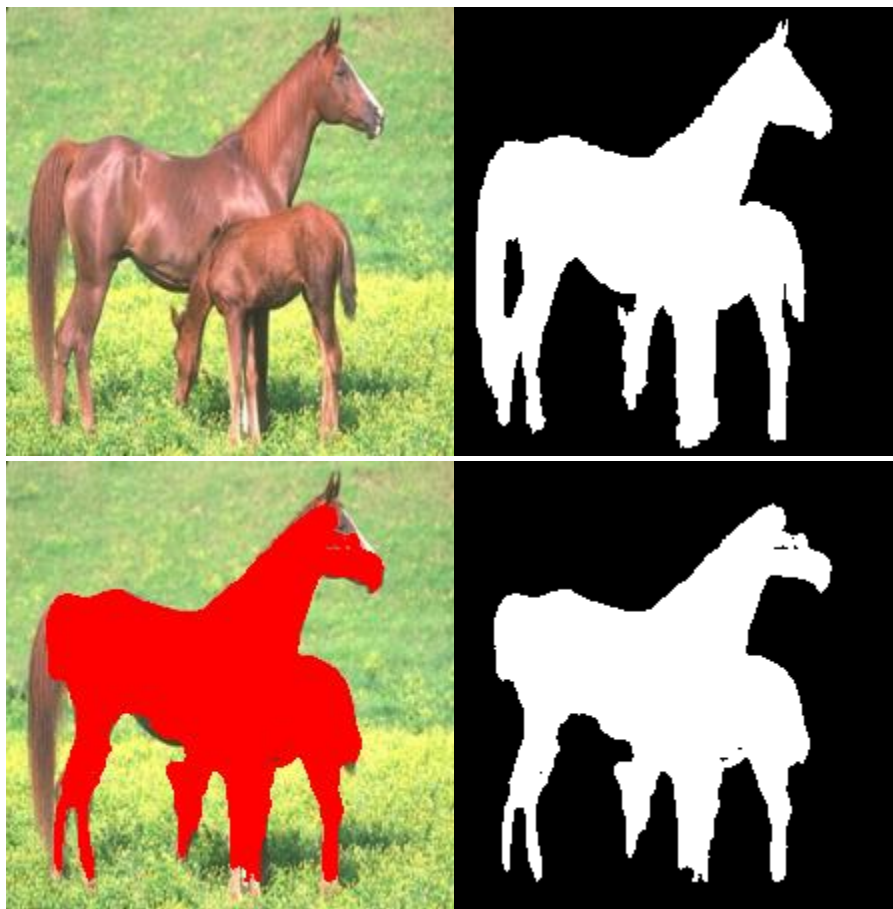


Figure 1: Example showing the original image (top-left), ground truth mask (top-right), predicted mask (bottom-right), and overlay (bottom-left).

The overlay images clearly show how the model highlights the main object. In most cases, the model successfully identifies the main region, but in some cases, it includes extra areas or misses smaller details.

10. Conclusion

In this project, I built a CNN-based model for salient object detection and tested its performance on a real dataset. The model was able to detect the main objects in images and produced reasonable results.

I also experimented with different numbers of training epochs and applied data augmentation to improve performance. While the results are not perfect, the model demonstrates a good understanding of the task and could be improved further with more advanced techniques.